

214. Shortest Palindrome

Hard · String · KMP · Rolling Hash

Approach — KMP Failure Function (LPS Array)

To make string `s` a palindrome by adding characters only to the front, we need to find the **longest palindromic prefix** of `s`. Whatever portion of `s` is not part of that prefix must be reversed and prepended.

This is found efficiently using the **KMP failure function** (LPS array). Construct a combined string `s + "*" + reverse(s)`. The separator `"*"` prevents the KMP match from crossing the boundary between `s` and its reverse. The LPS value at the last position of this combined string tells us the length of the longest prefix of `s` that is also a palindrome.

Key Insight: A prefix of `s` is a palindrome if and only if it equals its own reverse. By checking how much of `reverse(s)` matches a prefix of `s` (via KMP), we get exactly the length of the longest palindromic prefix. The remaining suffix of `s` is reversed and prepended to form the shortest palindrome.

Step-by-step Breakdown

Step 1 — Build the combined string

Reverse `s` into `str`, then form the combined string `s + "*" + str`. The `"*"` separator is a character guaranteed not to appear in `s`, ensuring the KMP match cannot overlap across the boundary of `s` and its reverse.

Step 2 — Compute the LPS (failure function) array

Run the KMP failure function on the combined string using two pointers `l` and `r`. When `str[l] == str[r]`, set `lps[r] = l+1` and advance both pointers. On mismatch, fall back using `l = lps[l-1]` if `l != 0`, otherwise set `lps[r] = 0` and advance `r`.

Step 3 — Extract the non-palindromic suffix

`lps.back()` gives the length of the longest palindromic prefix of `s`. Everything beyond that — `s.substr(lps.back())` — is the suffix that still needs to be mirrored. Extract it into `str`.

Step 4 — Prepend the reversed suffix

Reverse the extracted suffix and prepend it to s. The result `str + s` is the shortest palindrome that starts with the original string s.

Solution Code

```
1  class Solution {
2  public:
3      string shortestPalindrome(string s) {
4
5          string str = s;
6          reverse(str.begin(), str.end());
7          str = s + "*" + str;
8
9          vector<int> lps(str.size());
10         int l = 0, r = 1;
11         while(r < str.size())
12         {
13             if(str[l] == str[r])
14             {
15                 lps[r] = l+1;
16                 l++; r++;
17             }
18             else
19             {
20                 if(l != 0) l = lps[l-1];
21                 else lps[r] = 0, r++;
22             }
23         }
24         str = s.substr(lps.back());
25         reverse(str.begin(), str.end());
26         return str+s;
27     }
28 };
```

Complexity Analysis

TIME COMPLEXITY	SPACE COMPLEXITY
$O(n)$	$O(n)$
Building the combined string is $O(n)$. The KMP LPS computation runs in $O(n)$ over the combined string of length $2n+1$. Total: $O(n)$.	The combined string and LPS array each take $O(n)$ space, where n is the length of the input string s .